

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 6: CONDIÇÕES DE CORRIDA E GESTÃO DE ESCOPO COM OPENMP

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 27/08/2026.

1. EXPERIMENTO

Implemente em C a estimativa estocástica de π. Paralelize com "#pragma omp parallel for" e explique o resultado incorreto. Corrija a condição de corrida utilizando o "#pragma omp critical" e reestruturando com "#pragma omp parallel" seguido de "#pragma omp for" e aplicando as cláusulas "private, firstprivate, lastprivate e shared". Teste diferentes combinações e explique como cada cláusula afeta o comportamento do programa. Comente também como a cláusula "default(none)" pode ajudar a tornar o escopo mais claro em programas complexos.

2. CONCEITOS RELEVANTES

- **Diretiva *critical***: diretiva de sincronização que cria uma zona de exclusão mútua, garantindo que apenas uma thread por vez execute um bloco específico de código. Isso previne condições de corrida ao proteger variáveis compartilhadas (shared), sendo essencial quando múltiplas threads precisam acumular ou alterar um estado global de forma segura.
- **Cláusula *shared***:
  - Todas as threads criadas no bloco paralelo acessam e modificam exatamente a mesma posição de memória da variável original.
  - Se uma thread alterar o valor de uma variável shared, essa alteração é imediatamente visível para todas as outras threads.

- Sendo assim, o uso descuidado de variáveis compartilhadas pode gerar condições de corrida (data races), corrompendo os dados se duas threads tentarem modificar a mesma variável ao mesmo tempo sem sincronização. Por padrão, variáveis declaradas fora de um bloco paralelo costumam ser tratadas como “shared”.
- Por ser um estado global e concorrente, exige mecanismos de sincronização (como o critical) para evitar condições de corrida quando múltiplas threads tentam escrever simultaneamente.

❖ **Cláusula *private*:**
- Cria uma cópia local da variável para cada thread que entra no bloco paralelo.
- O valor inicial da variável dentro da thread é indefinido/lixo de memória (ela não herda o valor que tinha antes do bloco paralelo).
- As alterações feitas pela “thread X” na sua cópia privada não afetam a cópia da “thread Y” e nem a variável original fora do bloco paralelo.
- Quando o bloco paralelo termina, o valor original de fora continua exatamente o mesmo que era antes.

❖ **Cláusula *firstprivate*:**
- É uma evolução da cláusula private. Ela também cria uma cópia local para cada thread, mas com uma grande diferença na inicialização.
- Cada cópia local da thread é inicializada com o valor atual que a variável possuía fora do bloco paralelo (antes de ele começar).
- Perfeito para quando se precisa que as threads saibam qual era o estado inicial de uma variável, mas deseja que cada uma faça cálculos independentes a partir dali sem alterar o valor original global durante o processo.

❖ **Cláusula *lastprivate*:**
- Funciona em conjunto com loops paralelos (for). Ela torna a variável privada durante a execução das iterações, mas define um comportamento especial para o término.
- O valor da variável na última iteração lógica do loop (ou seja, quando o loop terminar de forma sequencial na última parte) é copiado de volta para a variável original fora do bloco.

❖ Cláusula default(none):
- Obriga o desenvolvedor a declarar explicitamente o escopo de compartilhamento (shared, private, firstprivate, etc.) de todas as variáveis declaradas fora da região paralela e utilizadas dentro dela.

➢ Se qualquer variável externa for usada dentro do bloco paralelo sem estar explicitamente listada em uma cláusula de escopo, o compilador gerará um erro de compilação (ex: "error: 'variavel' not specified in enclosing 'parallel'").

3. MÁQUINA UTILIZADA

Para a execução deste experimento, estou utilizando uma máquina equipada com um processador AMD Ryzen 7 5700X (8 núcleos físicos e 16 threads), que é o componente central para a avaliação das diretivas OpenMP. Suas 16 threads de hardware formam o ambiente físico onde a divisão do trabalho, as condições de corrida e o ganho de desempenho (speedup) se materializam durante a simulação simultânea de Monte Carlo. O sistema dispõe também de 32 GB de memória RAM e um SSD de 1TB, garantindo um ambiente isolado de gargalos de alocação de memória ou paginação pelo sistema operacional, o que assegura a precisão das métricas de tempo

4. ESTIMATIVA DE PI POR MONTE CARLO

A estimativa de π pelo método de Monte Carlo é uma técnica estatística que utiliza amostragens aleatórias para calcular resultados numéricos. O conceito baseia-se na probabilidade geométrica, relacionando a área de um círculo com a área de um quadrado que o circunscreve. Para simplificar a implementação computacional, o modelo analisa apenas o primeiro quadrante de um plano cartesiano.

Em um quadrado de lado "L = 1" com um quarto de círculo de raio r=1 inscrito nele.A área do quadrado é "A = $L^2 = 1^2 = 1$". A área do quarto de círculo é "A = πr^2 sobre 4". A razão geométrica entre a área do círculo e a área do quadrado é exatamente "πr^2 sobre 4".

A estimativa funciona através dos seguintes passos lógicos:

- **Geração de Pontos**: O algoritmo gera "N" pontos aleatórios com coordenadas (x, y), onde tanto "x" quanto "y" variam no intervalo contínuo de "0" a "1".
- **Teste de Inclusão:** Para cada ponto gerado, verifica-se se ele caiu dentro da área do quarto de círculo. Utilizando a equação da circunferência (e o Teorema de Pitágoras), um ponto está dentro do círculo se obedecer à condição "$x^2 + y^2 = 1$"
- **Contagem:** O programa conta o número total de pontos que satisfizeram a condição do passo 2 (vamos chamá-los de pontos internos, M).
- **Cálculo Final:** Como os pontos são distribuídos uniformemente, a probabilidade de um ponto cair dentro do círculo é igual à razão entre as

áreas. Portanto, “M sobre N” é igual a “A = πr^2 sobre 4” . O valor estimado de “π” é obtido isolando a variável: “π é aproximadamente 4 vezes M sobre N”

5. CÓDIGO

No primeiro programa implementado, conforme orientação do experimento, foi feita a estimativa de pi com “omp parallel for” sem cláusulas. Neste primeiro trecho (Figura 1), a diretiva divide as iterações entre as threads, mas gera uma grave condição de corrida na variável compartilhada “pontos_no_circulo”. Como a operação de incremento (++) não é atômica (ela exige ler, somar e gravar), múltiplas threads acessam e alteram o mesmo endereço de memória simultaneamente, o que faz com que sobreponham os valores umas das outras e percam milhares de contagens

Figura 1: Estimativa de pi com “omp parallel for”

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>

int main(int argc, char *argv[]) {
    long long total_pontos = 100000000LL; // Padrão: 100 milhões de pontos
    if (argc > 1) {
        total_pontos = atoll(argv[1]);
        if (total_pontos <= 0) {
            fprintf(stderr, "Erro: O numero de pontos deve ser positivo.\n");
            return 1;
        }
    }

    printf("Threads ativas : %d\n", omp_get_max_threads());
    printf("Total de pontos: %lld\n", total_pontos);

    long long pontos_no_circulo = 0; // Variável compartilhada não protegida!
    // Adicionando semente para a geração de números aleatórios
    srand((unsigned int)time(NULL)); // (sem semente os números aleatórios serão os mesmos em todas as execuções)
```

```
    double t_inicio = omp_get_wtime();

    #pragma omp parallel for
    for (long long i = 0; i < total_pontos; i++) {
        double x = (double)rand() / (double)RAND_MAX;
        double y = (double)rand() / (double)RAND_MAX;

        if (x * x + y * y <= 1.0) {
            // Múltiplas threads leem, incrementam e escrevem em 'pontos_no_circulo' ao
mesmo tempo.
            pontos_no_circulo++;
        }
    }

    double t_fim = omp_get_wtime();
    double tempo_total = t_fim - t_inicio;

    double pi_estimado = 4.0 * ((double)pontos_no_circulo / (double)total_pontos);
    double erro_absoluto = fabs(pi_estimado - M_PI);

    printf("Pontos no circulo : %lld / %lld (Contagens perdidas!)\n", pontos_no_circulo,
total_pontos);
    printf("Pi Estimado       : %.10f (Incorreto!)\n", pi_estimado);
    printf("Pi Real (M_PI)    : %.10f\n", M_PI);
    printf("Erro Absoluto     : %.10e\n", erro_absoluto);
    printf("Tempo de Execucao : %.6f segundos\n", tempo_total);

    return 0;
}
```

No segundo programa, conforme orientado a estrutura é dividida em duas etapas, com a criação da equipe de threads (#pragma omp parallel) e a posterior distribuição das iterações do laço (#pragma omp for). A condição de corrida é resolvida ao colocar a operação de incremento dentro de um bloco “#pragma omp critical”. Essa diretiva cria uma zona de exclusão mútua (como uma catraca), garantindo que apenas uma thread por vez acesse e altere a variável compartilhada “pontos_no_circulo”.

Figura 2: Incluindo “#pragma omp critical”

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>

int main(int argc, char *argv[]) {
    long long total_pontos = 100000000LL;
    if (argc > 1) {
        total_pontos = atoll(argv[1]);
        if (total_pontos <= 0) {
            fprintf(stderr, "Erro: O numero de pontos deve ser positivo.\n");
            return 1;
        }
    }

    printf("Threads ativas : %d\n", omp_get_max_threads());
    printf("Total de pontos: %lld\n", total_pontos);

    long long pontos_no_circulo = 0; // Variável compartilhada protegida por seção crítica
    srand((unsigned int)time(NULL));

    double t_inicio = omp_get_wtime();

    #pragma omp parallel // criação da equipe de threads
    {
        #pragma omp for // distribuição das iterações entre as threads
        for (long long i = 0; i < total_pontos; i++) {
            double x = (double)rand() / (double)RAND_MAX;
            double y = (double)rand() / (double)RAND_MAX;

            if (x * x + y * y <= 1.0) {
              // Seção crítica garantindo exclusão mútua para adição na variável compartilhada
                #pragma omp critical
                {
```

```
                pontos_no_circulo++;
            }
        }
    }
}

double t_fim = omp_get_wtime();
double tempo_total = t_fim - t_inicio;

double pi_estimado = 4.0 * ((double)pontos_no_circulo / (double)total_pontos);
double erro_absoluto = fabs(pi_estimado - M_PI);

printf("Pontos no circulo : %lld / %lld (CORRETO!)\n", pontos_no_circulo, total_pontos);
printf("Pi Estimado       : %.10f (CORRETO)\n", pi_estimado);
printf("Pi Real (M_PI)    : %.10f\n", M_PI);
printf("Erro Absoluto     : %.10e\n", erro_absoluto);
printf("Tempo de Execucao : %.6f segundos\n", tempo_total);

return 0;
}
```

No terceiro programa (figura 3), a cláusula private é introduzida para eliminar o gargalo de sincronização sem perder a exatidão. Em vez de travar o programa a cada acerto usando o critical no laço, a variável “contador_local” ganha uma cópia isolada na memória de cada thread. Como variáveis private iniciam com "lixo" de memória, elas são explicitamente zeradas no começo do bloco. Assim, cada thread soma seus próprios acertos de forma totalmente independente e veloz. A diretiva “#pragma omp critical” ainda é utilizada, mas agora apenas uma única vez por thread, no fim do processo, para somar o subtotal local à variável global pontos_no_circulo_total.

Figura 3: Incluindo cláusulas “shared” e “private”

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>
```

```c
#define PI 3.14159265358979323846

int main(int argc, char *argv[]) {
    long long total_pontos = 100000000LL; // 100 milhões de pontos
    if (argc > 1) {
        total_pontos = atoll(argv[1]);
        if (total_pontos <= 0) {
            fprintf(stderr, "Erro: O numero de pontos deve ser positivo.\n");
            return 1;
        }
    }

    printf("Threads ativas : %d\n", omp_get_max_threads());
    printf("Total de pontos: %lld\n", total_pontos);

    long long pontos_no_circulo_total = 0; // Compartilhada entre todas as threads
    long long contador_local;              // Será tornada 'private' para cada thread

    srand((unsigned int)time(NULL));

    double t_inicio = omp_get_wtime();

    // Região paralela que define o que é compartilhado e o que é privado
    #pragma omp parallel \
        shared(total_pontos, pontos_no_circulo_total) \
        private(contador_local)
    {
        // 'contador_local' é private, portanto inicia com lixo de memória
        // inicializando manualmente aqui dentro:
        contador_local = 0;

        double x, y, dist_sq; // Variáveis locais à thread (privadas)

        // Distribuindo as iterações do loop entre as threads
        #pragma omp for private(x, y, dist_sq)
        for (long long i = 0; i < total_pontos; i++) {
```

```
            x = (double)rand() / (double)RAND_MAX;
            y = (double)rand() / (double)RAND_MAX;
            dist_sq = x * x + y * y;

            if (dist_sq <= 1.0) {
                contador_local++; // Incremento local sem condição de corrida
            }
        }

        // Atualizando o total global apenas uma única vez por thread
        #pragma omp critical
        {
            pontos_no_circulo_total += contador_local;
        }
    }

    double t_fim = omp_get_wtime();
    double tempo_total = t_fim - t_inicio;

    double pi_estimado = 4.0 * ((double)pontos_no_circulo_total / (double)total_pontos);
    double erro_absoluto = fabs(pi_estimado - PI);

    printf("Pontos no circulo : %lld / %lld (Correto!)\n", pontos_no_circulo_total, total_pontos);
    printf("Pi Estimado       : %.10f (Correto!)\n", pi_estimado);
    printf("Pi Real (PI)      : %.10f\n", PI);
    printf("Erro Absoluto     : %.10e\n", erro_absoluto);
    printf("Tempo de Execucao : %.6f segundos\n", tempo_total);

    return 0;
}
```

Neste quarto trecho (figura 4), a cláusula “firstprivate” aprimora a versão anterior ao inicializar automaticamente a cópia local de cada thread com o valor original da variável (zero), dispensando a atribuição manual dentro do bloco paralelo.

Figura 4: Incluindo cláusulas “firstprivate”

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>

#define PI 3.14159265358979323846

int main(int argc, char *argv[]) {
    long long total_pontos = 100000000LL; // 100 milhões de pontos
    if (argc > 1) {
        total_pontos = atoll(argv[1]);
        if (total_pontos <= 0) {
            fprintf(stderr, "Erro: O numero de pontos deve ser positivo.\n");
            return 1;
        }
    }

    printf("Threads ativas : %d\n", omp_get_max_threads());
    printf("Total de pontos: %lld\n", total_pontos);

    long long pontos_no_circulo_total = 0; // Compartilhada
    long long contador_local = 0;          // Inicializada aqui e herdada via 'firstprivate'

    srand((unsigned int)time(NULL));

    double t_inicio = omp_get_wtime();

    // firstprivate: cada thread recebe sua própria cópia já inicializada com 0
    #pragma omp parallel \
        shared(total_pontos, pontos_no_circulo_total) \
        firstprivate(contador_local)
    {
            // 'contador_local' já inicia valendo 0 em todas as threads (sem precisar de inicialização manual)
        double x, y, dist_sq;
```

```
    // Distribuição das iterações do loop
    #pragma omp for private(x, y, dist_sq)
    for (long long i = 0; i < total_pontos; i++) {
        x = (double)rand() / (double)RAND_MAX;
        y = (double)rand() / (double)RAND_MAX;
        dist_sq = x * x + y * y;

        if (dist_sq <= 1.0) {
            contador_local++;
        }
    }

    //  Atualizando o total global apenas uma vez por thread
    #pragma omp critical
    {
        pontos_no_circulo_total += contador_local;
    }
  }

  double t_fim = omp_get_wtime();
  double tempo_total = t_fim - t_inicio;

  double pi_estimado = 4.0 * ((double)pontos_no_circulo_total / (double)total_pontos);
  double erro_absoluto = fabs(pi_estimado - PI);

  printf("Pontos no circulo : %lld / %lld (Correto!)\n", pontos_no_circulo_total, total_pontos);
  printf("Pi Estimado       : %.10f (Correto!)\n", pi_estimado);
  printf("Pi Real (PI)      : %.10f\n", PI);
  printf("Erro Absoluto     : %.10e\n", erro_absoluto);
  printf("Tempo de Execucao : %.6f segundos\n", tempo_total);

  return 0;
}
```

Por fim, a fim de verificar o comportamento do “lastprivate”, mesmo sabendo sua natureza (pegar o valor do último loop), o código abaixo (figura 5) também foi testado.

Figura 5: Incluindo cláusula “lastprivate”

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>

#define PI 3.14159265358979323846

int main(int argc, char *argv[]) {
    long long total_pontos = 100000000LL; // 100 milhões de pontos
    if (argc > 1) {
        total_pontos = atoll(argv[1]);
        if (total_pontos <= 0) {
            fprintf(stderr, "Erro: O numero de pontos deve ser positivo.\n");
            return 1;
        }
    }

    int num_threads = omp_get_max_threads();
    printf("Threads ativas : %d\n", num_threads);
    printf("Total de pontos: %lld\n", total_pontos);

    long long pontos_no_circulo = 0; // Inicializada com 0

    // Variáveis para registrar o último ponto sorteado no loop
    double ultimo_x = -1.0;
    double ultimo_y = -1.0;

    srand((unsigned int)time(NULL));

    double t_inicio = omp_get_wtime();

     // TENTATIVA: usar firstprivate para iniciar com 0 e lastprivate para "obter o total" ao
final
    #pragma omp parallel for \
        firstprivate(pontos_no_circulo) \
```

```
    lastprivate(pontos_no_circulo, ultimo_x, ultimo_y)
  for (long long i = 0; i < total_pontos; i++) {
    double x = (double)rand() / (double)RAND_MAX;
    double y = (double)rand() / (double)RAND_MAX;
    double dist_sq = x * x + y * y;

    if (dist_sq <= 1.0) {
      pontos_no_circulo++;
    }
  }

  double t_fim = omp_get_wtime();
  double tempo_total = t_fim - t_inicio;

  double pi_estimado = 4.0 * ((double)pontos_no_circulo / (double)total_pontos);
  double erro_absoluto = fabs(pi_estimado - PI);

  printf("\n--- RESULTADO DA TENTATIVA COM LASTPRIVATE ---\n");
  printf("Pontos no circulo : %lld / %lld (Incorreto! Apenas a contagem da ultima thread)\n",
      pontos_no_circulo, total_pontos);
  printf("Pi Estimado       : %.10f (Incorreto! Aproximadamente Pi / %d threads = %.4f)\n",
      pi_estimado, num_threads, PI / num_threads);
  printf("Pi Real (PI)      : %.10f\n", PI);
  printf("Erro Absoluto     : %.10e\n", erro_absoluto);
  printf("Tempo de Execucao : %.6f segundos\n", tempo_total);0

  return 0;
}
```

6. RESULTADOS

Para realizar os testes em cada estilo de código, foram executados todos os programas variando o número de threads. Para tal, em todas as execuções, foi utilizado 10 milhões como total de pontos.

Os resultados podem ser vistos nas tabelas abaixo.

Tabela 1: Valores de Pi calculados em função das threads

	1T	2T	4T	6T	8T	10T	12T	16T
Sequencial	3,141644	3,141644	3,141644	3,141644	3,141644	3,141644	3,141644	3,141644
“pragma parallel for”	3,141644	3,130971	3,134657	3,133163	3,13289	3,12993	3,12940	3,12903
“critical”	3,141870	3,141907	3,14157	3,141156	3,14182	3,14182	3,14170	3,14156
“shared private”	3,141848	3,141963	3,14191	3,141723	3,14195	3,14135	3,14110	3,14082
“firstprivate”	3,140694	3,140966	3,14184	3,141218	3,14152	3,14153	3,14159	3,14157
“lastprivate”	3,141788	1,570701	0,78553	5,233820	0,392762	0,314215	0,26178	0,19624

Figura 6: Valores de Pi calculados em função das threads

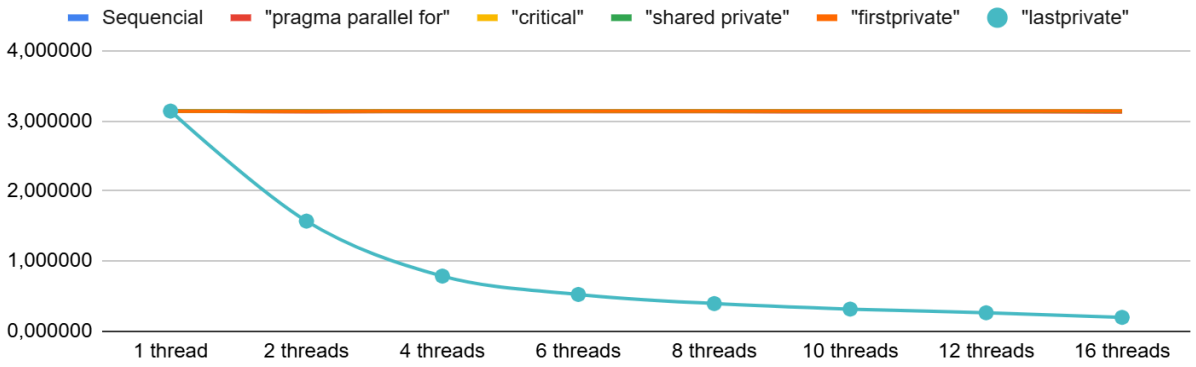


Tabela 2: Valores de erro absoluto de Pi calculados em função das threads

	1T	2T	4T	6T	8T	10T	12T	16T
Sequencial	0,000051	0,000051	0,0000513464	0,000051	0,0000513464	0,0000513464	0,0000513464	0,0000513464
“pragma parallel for”	0,000051	0,010621	0,0069358536	0,0084298536	0,0087014536	0,0116578536	0,0121906536	0,0125666536

“critical”	0,000278	0,000314	0,0000274536	0,00044	0,0002321464	0,0002297464	0,0001037464	0,0000338536
“shared private”	0,000255	0,0003701464	0,0003213464	0,00013	0,0003597464	0,0002394536	0,0004934536	0,0007682536
“firstprivate”	0,000899	0,0006270536	0,0002493464	0,00038	0,0000702536	0,0000598536	0,0000017464	0,0000270536
“lastprivate”	0,000196	1,570891454	2,356059054	2,618210654	2,748830654	2,827377454	2,879811854	2,945349054

Figura 7: Valores de erro absoluto de Pi calculados em função das threads (exceto “lastprivate”)

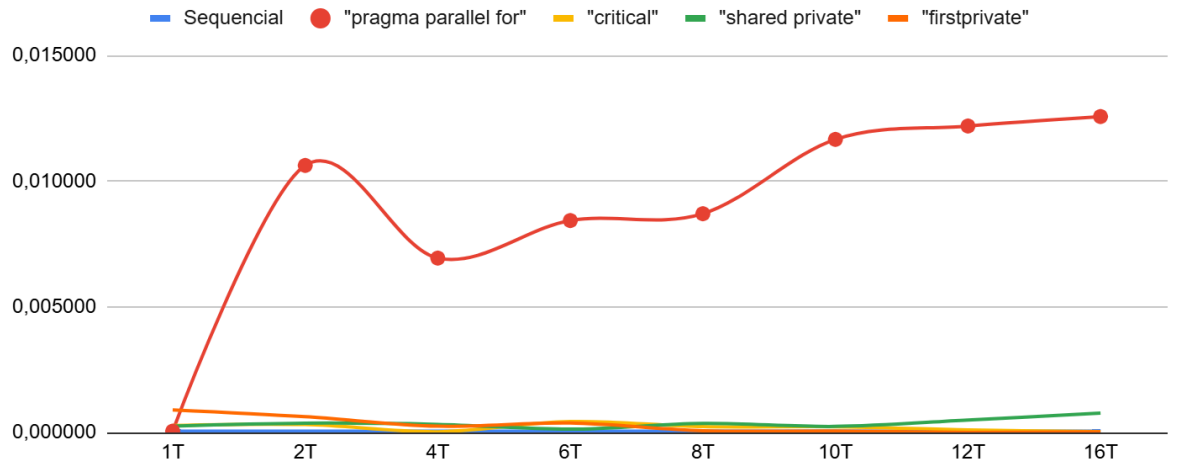
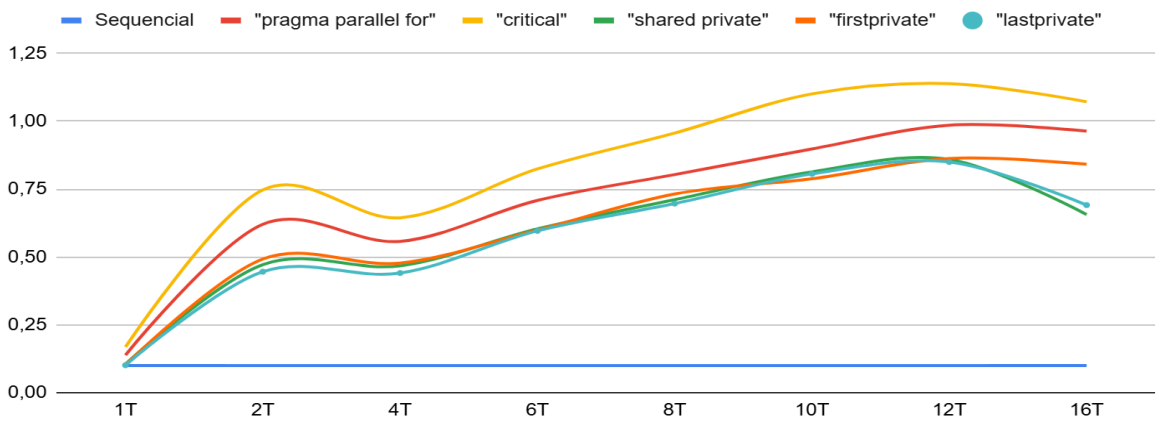


Tabela 3: Valores de tempo (s) medidos em função das threads

	1T	**2T**	**4T**	**6T**	**8T**	**10T**	**12T**	**16T**
Sequencial	0,100449	0,100449	0,100449	0,100449	0,100449	0,100449	0,100449	0,100449
“pragma parallel for”	0,139063	0,620174	0,557420	0,708135	0,803327	0,897703	0,985198	0,963597
“critical”	0,168928	0,746692	0,644294	0,824025	0,956301	1,100325	1,138214	1,071356
“shared private”	0,106045	0,471894	0,467373	0,603313	0,711019	0,813428	0,859202	0,656307
“firstprivate”	0,103022	0,492349	0,477556	0,599482	0,732037	0,788159	0,862593	0,841388

“lastprivate”	0,101825	0,445919	0,441045	0,596199	0,696638	0,806049	0,84968	0,691108

Figura 8: Valores de tempo (s) medidos em função das threads



A análise dos resultados consolidados na tabela revela comportamentos fundamentais sobre concorrência, evidenciando que a simples adição de diretivas paralelas não garante aceleração e exige gestão estrita de memória compartilhada.

Pode-se perceber que:

- A execução sequencial estabelece uma linha de base exata e rápida. Ao introduzir o #pragma omp parallel for sem tratamento de escopo, a variável de soma torna-se implicitamente compartilhada. A concorrência das 16 threads gera colisões de memória (condição de corrida), resultando em perdas massivas de contagem e na degradação severa da estimativa de “pi” conforme o número de threads aumenta.
- A aplicação do “lastprivate” provou-se incompatível com algoritmos de soma global. Como o lastprivate copia apenas o estado da thread que executa a última iteração do laço, o processamento das outras 15 threads foi sumariamente descartado, resultando em um erro matemático extremo.
- O uso puro do “#pragma omp critical” dentro do laço corrige os erros de memória, mas força o processador a criar uma fila de acesso a cada acerto, multiplicando o tempo de execução por dez. A adoção das cláusulas private e firstprivate aliviou esse gargalo arquitetônico. Ao permitir que cada thread conte os acertos em variáveis locais isoladas e acione a zona crítica apenas uma vez no fim do processamento, o overhead de travamento caiu de milhões de vezes para apenas 16 vezes, refletindo-se em uma leve melhora no tempo de execução.
- Outro dado relevante no relatório é que nenhuma versão paralela superou a velocidade do código sequencial (~0,10s). Isso pode ser justificado pelo uso da

função padrão “rand()” do C. Por não ser segura para a concorrência (thread-safe), ela possui mecanismos internos de travamento para proteger sua semente geradora. Quando múltiplas threads tentam chamá-la simultaneamente, geram um engarrafamento oculto que anula todo o ganho de processamento distribuído.

7. CONSIDERAÇÕES FINAIS

Por fim, pode-se compreender que a escolha das cláusulas de dados dita a segurança e a eficiência da paralelização. O escopo “shared” expõe o código a condições de corrida caso as variáveis globais não sejam protegidas, enquanto as cláusulas “private” e “firstprivate” eliminam essa contenção ao isolar a contagem em cópias locais na memória de cada thread, com o diferencial de que a segunda já inicializa a variável automaticamente. A cláusula “lastprivate”, por outro lado, corrompe este algoritmo de soma ao salvar apenas a última iteração do laço e descartar o restante do trabalho paralelo.

Para evitar esses erros e ambiguidades em sistemas complexos, a adoção da diretiva default(none) é fundamental. É ela quem desabilita as decisões automáticas do OpenMP e obriga o desenvolvedor a declarar explicitamente o escopo de cada variável, prevenindo compartilhamentos acidentais e tornando a arquitetura de memória cristalina.